

プログラミング言語論

第1回

篠埜 功

講義のスケジュール

- 講義6回、期末試験
- 成績評価
 - 期末試験 90点満点
 - 小テスト 10点満点
 - 期末試験F点、小テストS点のとき、
 $S + F * (100 - S) / 90$ 点を合計得点とする。

連絡先

- 篠埜 功

居室: 豊洲校舎 14階 14K32

E-mail: sasano@sic.shibaura-it.ac.jp

web: <http://www.sic.shibaura-it.ac.jp/~sasano/index-j.html>

講義用ページ: 上記webページから講義情報ページへリンクを張っている。

- SA、TAはいない。

機械語(machine language)

- 機械語(machine language)はもっとも低レベルの言語。コンピュータが直接解釈実行する。
 - 機械語は最初、コード(code)と呼ばれた。(今日では高級言語のプログラムのこともコードと言う。)
- フォンノイマンマシン(von Neumann machine)
 - 1946年にプリンストン高等研究所で設計。
 - Turing machineをランダムアクセスマシンにし、入出力装置を追加したものに相当。

機械語(machine language)

数字の列であらわされる。

[1946年に設計されたvon Neumann machineの機械語の断片例]

00000010101111001010

00000010111111001000

00000011001110101000

(10番地と11番地の値を足し、その結果を12番地に保存する)

現代のコンピュータも原理的にはvon Neumann machineと同じであり、“von Neumann architecture”のコンピュータと言われる。

アセンブリ言語(assembly language)は機械語の命令を記号で表す。

プログラミング言語

プログラミング言語は特定の機械に依存しないことが(通常は)望まれる(高水準言語)。

プログラミング言語が持つべき性質

➤ 高水準の記述能力

- プログラムの論理構造を簡潔に記述

➤ 厳密な意味の定義

- その言語で記述可能なすべてのプログラムの意味(動作)を完全に規定

➤ 効率的実装

- その言語で記述可能なすべてのプログラムを、効率のよい機械語に変換

高水準言語の利点

機械語、アセンブリ言語はほとんどすべての領域において高水準言語にとってかわられた。

- 人間にとって読みやすい
- 特定の機械に依存しない (portable)
- ライブラリが使える
- 構文チェック、型チェックなどの検査機構がある

例: C言語

- 1973年に開発, UNIXのkernel(最初アセンブリ言語で書かれていた)をCで書き換えた
- アセンブリ言語で書かれていたころより修正や、新しいデバイスの追加に対する拡張などがはるかに容易になった
- 異なるハードウェア(PDP-11以外)に対するUNIX OSも、コードの大部分を変えることなく作れた。

プログラミング言語の分類

プログラミング言語は計算モデルにより以下のように分類される。

- 命令型言語(imperative language)あるいは手続き型言語(procedural language)
- 関数型言語(functional language)
- オブジェクト指向言語(object oriented language)
- 論理型言語(logic programming language)

言語が提供するもの(1)

- 計算モデル(前ページ参照)
- データ型(とその操作)
 - (例) C言語はint型, double型などの基本型を提供する。またC言語は構造体(レコード)、ポインタ、共用体等、データを組み合わせて大きな構造のデータを作る仕組みを提供する。int型, string型, double型などの基本型のデータをそれらで組み合わせることにより、複雑な構造のデータを組み立てることができる。また、構造体等に対し、その部分構造を得る操作関数(構造体のメンバ演算子など)が提供される。

言語が提供するもの(2)

- 抽象化機構
 - (例1) 命令型言語における変数
 - メモリ番地を抽象化している
 - コンパイル時、実行時にメモリ番地へ具体化される
 - (例2) 命令型言語における関数
 - 計算パターンを抽象化している
 - 関数呼び出しにより具体化(仮引数に実引数の値を代入)
 - (例3) Cにおけるgoto文、Javaにおける例外
 - Jumpを抽象化、具体化
 - (例4)[参考] 関数型言語における多相型
 - 型を抽象化、具体化
- 検査機構
 - (例) 構文チェック、型検査
 - コンパイル時に構文エラー、型エラーを検出

プログラミング言語の構文

- (例) BNF記法による数字列言語の構文定義

$\langle d \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$\langle \text{digit_seq} \rangle ::= \langle d \rangle$

$\mid \langle d \rangle \langle \text{digit_seq} \rangle$

$\langle \text{real-number} \rangle ::= \langle \text{digit_seq} \rangle . \langle \text{digit_seq} \rangle$

通常、プログラミング言語の文法は
文脈自由文法(context-free grammar)に属する。
文脈自由文法に属する文法はBNF記法で記述できる。
(BNF記法は文脈自由文法を簡潔に記述する記法)

プログラミング言語の意味

(例) 日付を表す言語の構文

`<date> ::= <d><d> / <d><d> / <d><d><d><d>`

01/02/2001

アメリカでは2001年1月2日を表す。

2001年2月1日を表す国もある。

プログラミング言語は構文と意味を定義することにより定義される。

プログラミング言語の定義、説明

- チュートリアル(Tutorial)
 - 言語の概略を紹介。
- レファレンスマニュアル(Reference manual)
 - 構文と意味を記述。BNFによる構文定義と英語などの自然言語による意味の説明。
- 形式的定義(Formal definition)
 - 英語や日本語などの自然言語ではなく、操作的意味論、表示の意味論、公理の意味論など、形式的な議論に耐える意味の記述。

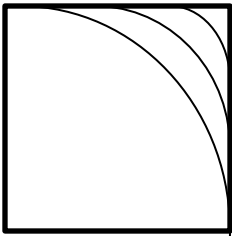
簡単な言語の例---Little Quilt



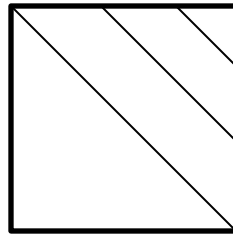
Quiltの例

Little Quilt言語

- 2つの基本図形(正方形)を組み合わせて quiltを作成する言語



a



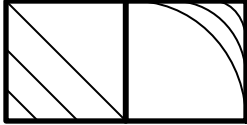
b

Little Quilt言語の式の定義

$\langle \text{exp} \rangle ::= a$
 $| b$
 $| \text{turn } (\langle \text{exp} \rangle)$
 $| \text{sew } (\langle \text{exp} \rangle, \langle \text{exp} \rangle)$

$\text{turn } (e)$ --- キルト e を90度右回転させたキルトを表す。
 $\text{sew } (e_1, e_2)$ --- 高さが同じキルト e_1, e_2 を左右に並べ、縫い合わせる（左が e_1 、右が e_2 ）。

Little Quilt言語の式の例

式	quilt
b	
turn (b)	
turn (turn (b))	
a	
sew (turn (turn (b))), a)	

演習問題1

以下の式が表わすQuiltを図示せよ。

$\text{turn}(\text{sew}(\text{turn}(b), \text{turn}(b)))$

関数宣言の導入

fun unturn (x) = turn (turn (turn (x)))

左回転操作

fun pile (x,y) = unturn (sew (turn (y), turn (x)))

幅の等しいキルト xとyを上下に並べて縫い合わせる(上がx, 下がy)

このように、よく使う計算パターンに名前を付けることができるると便利。

関数宣言の構文

fun <name> (<formals>) = <exp>

<formals> ::= <name> | <name>, <formals>

ただし、<exp>の定義に<name>と関数適用式を追加する。(後述)

局所宣言(let式)の導入

(例)

let

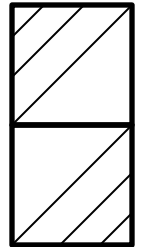
fun unturn (x) = turn (turn (turn (x)))

fun pile (x,y) = unturn (sew (turn (y), turn (x)))

in

pile (unturn (b), turn (b))

end



let式の構文

let <decls> **in** <exp> **end**

<decls>の定義は後で行う。

<decls>で宣言された関数名の有効範囲は、<decls>内のその宣言以降およびinとendの間。ただし、その範囲内で同じ名前が導入された場合はその名前の有効範囲を除いた部分。

演習問題2

以下の式が表わすQuiltを図示せよ。

```
let
  fun f (x) = turn (turn (x))
in
  f (f (b))
end
```

値に名前を付ける構文の導入

(例)

let

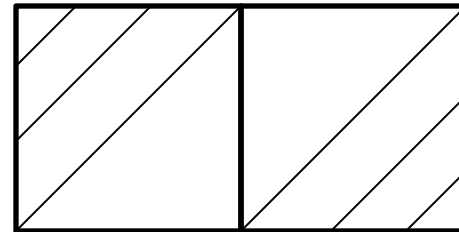
val x = unturn (b)

val y = turn (b)

in

sew (x,y)

end



値に名前を付ける構文 (value declaration)

val <name> = <exp>

この構文についても、名前の有効範囲は関数宣言の場合と同じ。

少し大きな例

let

fun unturn (x) = turn (turn (turn (x)))

fun pile (x,y) = unturn (sew (turn (y), turn (x)))

val aa = pile (a, turn (turn (a)))

val bb = pile (unturn (b), turn (b))

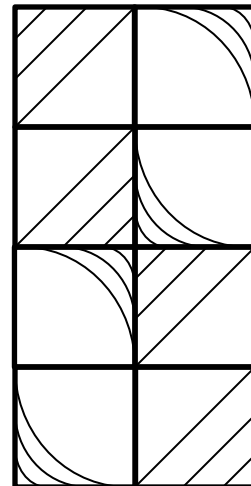
val p = sew (bb, aa)

val q = sew (aa, bb)

in

pile (p,q)

end



Little Quilt言語の構文定義

$\langle \text{exp} \rangle ::= a \mid b \mid \langle \text{name} \rangle \mid \langle \text{name} \rangle (\langle \text{actuals} \rangle)$
 $\mid \text{turn} (\langle \text{exp} \rangle) \mid \text{sew} (\langle \text{exp} \rangle, \langle \text{exp} \rangle)$
 $\mid \text{let } \langle \text{decls} \rangle \text{ in } \langle \text{exp} \rangle \text{ end}$
 $\langle \text{actuals} \rangle ::= \langle \text{exp} \rangle \mid \langle \text{exp} \rangle , \langle \text{actuals} \rangle$
 $\langle \text{decls} \rangle ::= \langle \text{decl} \rangle \mid \langle \text{decl} \rangle \langle \text{decls} \rangle$
 $\langle \text{decl} \rangle ::= \text{fun } \langle \text{name} \rangle (\langle \text{formals} \rangle) = \langle \text{exp} \rangle$
 $\mid \text{val } \langle \text{name} \rangle = \langle \text{exp} \rangle$
 $\langle \text{formals} \rangle ::= \langle \text{name} \rangle \mid \langle \text{name} \rangle , \langle \text{formals} \rangle$

$\langle \text{name} \rangle$ は文字列など。通常、字句解析で処理する。

$\langle \text{name} \rangle ::= \langle \text{c} \rangle \mid \langle \text{c} \rangle \langle \text{name} \rangle$

$\langle \text{c} \rangle ::= a \mid b \mid c \mid d \mid e \dots$ などで定義できる。